

ESTADO DEL ARTE

Hermes Stack frente al State of the Art 2026

Comparativa exhaustiva por categoría

Análisis de cada tecnología del Hermes Stack frente a las alternativas dominantes de la industria — con veredicto razonado

Versión:	1.0 · Documento canónico SOTA
Fecha:	28 de mayo de 2026
Categorías analizadas:	13 + síntesis global
Alternativas evaluadas:	50+ herramientas
Benchmarks incluidos:	latencia, costo, MTEB, ecosistema
Horizonte temporal:	2026-2027
Documento compañero:	ficha_tecnica_hermes.pdf

Análisis técnico independiente — sin patrocinio de proveedores
Compilado el 28 de mayo de 2026 con XeLaTeX · DejaVu Fonts

github.com/erikjosehrndz-crypto/hermes-stack

Índice de contenidos

1. Introducción y metodología	5
1.1. Propósito del documento	5
1.2. Metodología de comparación	5
1.3. Escala de puntuación	5
1.4. Categorías cubiertas	5
1.5. Marco temporal	5
1.6. Sesgos declarados	6
2. Categoría 1: Orquestación de agentes IA	7
2.1. El problema	7
2.2. Alternativas evaluadas	7
2.3. Tabla comparativa	8
2.4. Tendencia 2026-2027	9
2.5. Recomendación para Hermes	9
3. Categoría 2: LLM routing y gateway	10
3.1. El problema	10
3.2. Alternativas evaluadas	10
3.3. Tabla comparativa	11
3.4. Benchmark de costo (\$/1M tokens, mayo 2026)	11
3.5. Recomendación	11
4. Categoría 3: Speech-to-Text (STT)	12
4.1. El problema	12
4.2. Alternativas evaluadas	12
4.3. Tabla comparativa	13
4.4. Benchmark de latencia (frase de 3 s, español)	13
4.5. Recomendación	13
5. Categoría 4: Text-to-Speech (TTS)	14
5.1. El problema	14
5.2. Alternativas evaluadas	14
5.3. Tabla comparativa	15
5.4. Benchmark de latencia hasta primer chunk de audio	16
5.5. Recomendación	16
6. Categoría 5: Vector databases	17
6.1. El problema	17
6.2. Alternativas evaluadas	17
6.3. Tabla comparativa	18
6.4. Benchmark — query p50 (50k vectores, 1024 dims, cosine)	18
6.5. Recomendación	18

7. Categoría 6: Graph databases para memoria	19
7.1. El problema	19
7.2. Alternativas evaluadas	19
7.3. Tabla comparativa	20
7.4. Patrón de acceso multi-proceso	20
7.5. Recomendación	20
8. Categoría 7: Embedding models	21
8.1. El problema	21
8.2. Alternativas evaluadas	21
8.3. Tabla comparativa	22
8.4. Benchmark MTEB (Spanish subset, 2026-05)	22
8.5. Recomendación	22
9. Categoría 8: Caché y message queue	23
9.1. El problema	23
9.2. Alternativas evaluadas	23
9.3. Tabla comparativa	24
9.4. Benchmark — ops/s (single instance, GET/SET, valor 256 B)	24
9.5. Recomendación	24
10Categoría 9: Reverse proxy / TLS	25
10.1El problema	25
10.2Alternativas evaluadas	25
10.3Tabla comparativa	26
10.4Comparativa de configuración para 7 subdominios con TLS	26
10.5Recomendación	27
11Categoría 10: Observabilidad	28
11.1El problema	28
11.2Alternativas evaluadas	28
11.3Tabla comparativa	29
11.4Plan de evolución	29
11.5Recomendación	29
12Categoría 11: Frontend para AI apps	30
12.1El problema	30
12.2Alternativas evaluadas	30
12.3Tabla comparativa	31
12.4Patrones críticos aplicados en Hermes Website	31
12.5Recomendación	31
13Categoría 12: Protocolos de agentes (MCP)	32
13.1El problema	32
13.2Alternativas evaluadas	32
13.3Tabla comparativa	33
13.4Patrón FastMCP típico	33
13.5Configuración cliente (Claude Code)	33
13.6Recomendación	34

14	Categoría 13: Containerización y orquestación	35
14.1	El problema	35
14.2	Alternativas evaluadas	35
14.3	Tabla comparativa	36
14.4	Comparativa de complejidad operativa	36
14.5	Recomendación	36
15	Síntesis global — Hermes vs SOTA	37
15.1	Puntuación consolidada por categoría	37
15.2	Radar global Hermes vs SOTA	38
15.3	Conclusiones de la síntesis	38
16	Tendencias 2026-2027	39
16.1	Cinco tendencias dominantes	39
16.2	Línea de tiempo del roadmap SOTA para Hermes	40
16.3	Mejoras de mayor impacto vs esfuerzo	40
17	Roadmap de adopción SOTA — detalle	41
17.1	Corto plazo (Q3-Q4 2026)	41
17.2	Medio plazo (Q1-Q2 2027)	41
17.3	Largo plazo (H2 2027)	42
18	Conclusiones	43
18.1	Veredicto general	43
18.2	Las 4 decisiones que distinguen a Hermes positivamente	43
18.3	Las 3 áreas con gap respecto al SOTA	43
18.4	Las 2 apuestas a largo plazo	43
18.5	Mensaje de cierre	43
19	Bibliografía y fuentes	44
19.1	Documentación oficial	44
19.2	Benchmarks y evaluaciones	45
19.3	Papers académicos relevantes	45
19.4	Blogs y posts de referencia	45
19.5	Documentación interna del Hermes Stack	45
20	Apéndice — benchmarks de referencia	46
20.1	Latencia E2E del pipeline de voz (50 muestras)	46
20.2	Distribución de cache hits LiteLLM (últimas 24 h)	46
20.3	Consumo de recursos por contenedor (estable, 24 h)	46
20.4	Costo operacional (estimación mensual)	47
20.5	Disponibilidad histórica	47

1. Introducción y metodología

1.1. Propósito del documento

Este documento compara cada decisión tecnológica del **Hermes Stack** contra las alternativas dominantes del **estado del arte** en 2026. El objetivo no es justificar las decisiones existentes, sino evaluarlas críticamente para identificar:

- 1. Qué decisiones son sólidas y deben mantenerse.
- 2. Dónde el stack se desvía del SOTA y si esa desviación tiene justificación válida.
- 3. Qué oportunidades de mejora existen a corto, medio y largo plazo.

1.2. Metodología de comparación

Para cada una de las **13 categorías tecnológicas** se aplica el siguiente esquema:

- 1. **Identificación de alternativas:** las 4-5 opciones más relevantes del SOTA 2026.
- 2. **Dimensiones de evaluación:** entre 5 y 8 dimensiones (latencia, costo, ecosistema, etc.).
- 3. **Tabla comparativa:** con escala visual ●●●○○ (de 1 a 5).
- 4. **Análisis cualitativo:** fortalezas y debilidades de cada opción.
- 5. **Posición de Hermes:** qué eligió el stack y por qué — con veredicto explícito.

1.3. Escala de puntuación

Símbolo	Puntuación	Significado
●●●●●	5 / 5	Excelente — referencia de la industria
●●●●○	4 / 5	Muy bueno — pocas pegas relevantes
●●●○○	3 / 5	Aceptable — varias limitaciones conocidas
●●○○○	2 / 5	Deficiente — gap respecto al SOTA
●○○○○	1 / 5	Obsoleto / no recomendable

1.4. Categorías cubiertas

#	Categoría	#	Categoría
1	Orquestación de agentes IA	8	Caché y message queue
2	LLM routing y gateway	9	Reverse proxy / TLS
3	Speech-to-Text (STT)	10	Observabilidad
4	Text-to-Speech (TTS)	11	Frontend para AI apps
5	Vector databases	12	Protocolos de agentes (MCP)
6	Graph databases	13	Containerización y orquestación
7	Embedding models	—	Síntesis global + tendencias 2026-2027

1.5. Marco temporal

Toda la información refleja el **estado del ecosistema en mayo de 2026**. Cada categoría se actualiza al ritmo que se mueven sus principales actores (LLMs cambian mensualmente; reverse proxies anualmente), por lo que el documento incluye una sección final de **tendencias proyectadas 2026-2027** para evitar quedar congelado.

1.6. Sesgos declarados

- Sesgo a favor de **open-source** sobre SaaS cerrado (alineado con principios del stack).
- Sesgo a favor de soluciones **embedded** / **single-host** sobre clusters distribuidos (alineado con el tamaño del workload).
- Sesgo a favor de protocolos **abiertos** (MCP, OpenAI-compatible) sobre APIs propietarias.

Estos sesgos se hacen explícitos para que el lector pueda recalibrar si tiene restricciones diferentes (por ejemplo, una empresa con compliance Enterprise SaaS-first).

2. Categoría 1: Orquestación de agentes IA

2.1. El problema

Un agente IA conversacional necesita coordinar: contexto de la conversación, llamadas al LLM, ejecución de herramientas, memoria persistente, manejo de errores y streaming de respuestas. Existen frameworks que abstraen esta orquestación (LangChain, AutoGen, CrewAI, Pydantic-AI, DSPy) y la opción de hacerlo **manual** con Python async.

2.2. Alternativas evaluadas

LangChain / LangGraph

- Framework dominante por adopción y ecosistema (integraciones con 200+ proveedores).
- **Punto débil:** sobrecarga arquitectónica fuerte; cada update minor rompe APIs (versiones 0.1 → 0.2 → 0.3 reescrituras parciales).
- **Ideal para:** POCs rápidos y exploración multi-proveedor.

AutoGen (Microsoft)

- Framework multi-agente con foco en agent-to-agent conversation.
- **Punto débil:** overhead de mensajería; latencia notablemente mayor.
- **Ideal para:** workflows con N agentes que conversan entre sí.

CrewAI

- Framework con metáfora de equipo (crew, roles, tareas).
- **Punto débil:** abstracciones rígidas; difícil personalizar el control loop.
- **Ideal para:** procesos secuenciales bien definidos.

Pydantic-AI

- Framework minimalista basado en validación Pydantic.
- **Punto fuerte:** type safety, structured outputs, dependencias mínimas.
- **Punto débil:** ecosistema joven; aún sin patrones probados para streaming WS.

DSPy (Stanford)

- Framework para **optimización automática** de prompts (compila programa LLM).
- **Punto fuerte:** convierte prompts en programas que se optimizan via teleprompter.
- **Punto débil:** curva de aprendizaje empinada; ideal para investigación, no producción inicial.

Python async manual (aiohttp + asyncio)

- Sin framework — todo escrito a mano con aiohttp, websockets, asyncio.
- **Punto fuerte:** control total, sin sobrecarga, sin breaking changes externos.
- **Punto débil:** hay que escribir patrones que en LangChain vienen gratis (retry, fallback, observability).

2.3. Tabla comparativa

Framework	Ecosis.	Latencia	Control	Estabil.	Streaming	Veredicto
LangChain	●●●●●	●●●○○	●●○○○	●●○○○	●●●○○	adopción alta, breaking changes frecuentes
AutoGen	●●●○○	●●○○○	●●●○○	●●●○○	●●○○○	ideal multi-agent, lento single-agent
CrewAI	●●●○○	●●●○○	●●○○○	●●●○○	●●○○○	rígido pero estable
Pydantic-AI	●●●○○	●●●●○	●●●●○	●●●●●	●●●○○	moderno, joven
DSPy	●●●○○	●●○○○	●●●●○	●●●○○	●●○○○	investigación, no producción inicial
aiohttp manual	●●○○○	●●●●●	●●●●●	●●●●●	●●●●●	control total, deuda inicial

◆ Posición Hermes: Hermes usa Python async manual con aiohttp + websockets

El stack rechazó LangChain/CrewAI por la sobrecarga arquitectónica y los breaking changes; eligió Python puro porque controla el voice pipeline al milisegundo y porque la lógica de skills es lo suficientemente acotada para no necesitar abstracciones de framework.

Trade-off aceptado: más código inicial (cliente HTTP, retry, métricas) a cambio de cero deuda de framework. **Riesgo principal:** reimplementar capacidades que un framework regalaría — mitigado al limitar el agente a tres dominios (voz, LLM, memoria) y no expandirse a workflows complejos.

2.4. Tendencia 2026-2027

La mayoría de proyectos serios están migrando de LangChain a frameworks más finos (**Pydantic-AI**, **LlamaIndex**) o a Python puro. La adopción de **Model Context Protocol (MCP)** reduce la presión por un framework — el protocolo estandariza la interfaz tool ↔ agent, eliminando una capa de abstracción.

2.5. Recomendación para Hermes

Mantener Python async manual. El stack ya tiene los patrones críticos (ClientSession compartida, TTLCache para dedup, orjson, Prometheus metrics) y migrar a un framework introduciría riesgo sin ganancia clara. Cuando el agente crezca a 10+ skills complejos, evaluar **Pydantic-AI** como migración incremental por su afinidad estructural.

3. Categoría 2: LLM routing y gateway

3.1. El problema

Un sistema con múltiples proveedores LLM (OpenAI, Anthropic, Google, Meta) necesita una capa de abstracción que: (1) normalice las APIs, (2) cachee respuestas, (3) haga fallback ante errores, (4) trackee costos y latencia, (5) exponga métricas. Hacerlo a mano duplica trabajo en cada llamada.

3.2. Alternativas evaluadas

LiteLLM (BerriAI)

- Open-source, MIT license, escrito en Python.
- 200+ proveedores LLM normalizados a la API de OpenAI.
- Routing strategies: latency-based, weighted, least-busy.
- Caché Redis nativa, Prometheus metrics, fallback chains.
- Self-hostable como proxy proxy o como biblioteca.

Portkey

- SaaS gestionado con gateway open-source.
- Buen UI para observabilidad y gestión de costos.
- **Punto débil:** core SaaS (tier gratuito limitado).

OpenRouter directo

- Agregador de APIs LLM con una sola API key.
- **Punto débil:** sin caché propia, sin métricas, sin fallback local; añade un hop a cada llamada.

Ollama

- Runtime para LLMs locales (Llama, Mistral, Phi, etc.) con API HTTP.
- **Punto fuerte:** privacidad absoluta, costo \$0.
- **Punto débil:** requiere GPU; modelos open-weight aún por detrás de GPT-4o/Claude en muchas tareas.

vLLM

- Servidor de alto rendimiento para LLMs open-weight (PagedAttention, continuous batching).
- **Punto fuerte:** throughput máximo, ideal para cargas internas.
- **Punto débil:** dedicado a modelos open; sin abstracción multi-proveedor.

3.3. Tabla comparativa

Gateway	Provs.	Cache	Fallback	Metrics	Costo	Veredicto
LiteLLM	●●●●●	●●●●●	●●●●●	●●●●●	●●●●●	estándar de facto self-hosted
Portkey	●●●●●	●●●●○	●●●●○	●●●●●	●●●○○	SaaS con buen UI
OpenRouter	●●●●○	●○○○○	●○○○○	●●○○○	●●●●○	solo agregación, sin lógica
Ollama	●●○○○	●○○○○	●○○○○	●●○○○	●●●●●	local-only, requiere GPU
vLLM	●●○○○	●●○○○	●○○○○	●●●○○	●●●●●	throughput máximo open-weight

◆ **Posición Hermes:** Hermes usa LiteLLM como router principal

LiteLLM es la elección óptima: cubre las 5 dimensiones críticas (proveedores, cache, fallback, métricas, costo) sin sacrificar control. La caché Redis integrada ahorra 33 % de llamadas; el fallback automático entre Gemini Flash, GPT-4o-mini y Llama 3.1 70B mantiene SLO ante caídas de proveedor; las métricas Prometheus se integran sin esfuerzo.

Configuración actual: 5 modelos en 2 tiers, routing por latencia, cache TTL 3600s. Coste licencia: \$0 (MIT). El proxy 9router cumple un rol complementario para conexiones MCP-stdio, no compite con LiteLLM.

3.4. Benchmark de costo (\$/1M tokens, mayo 2026)

Modelo	Input	Output	Latencia típica
Gemini 2.5 Flash	\$0.075	\$0.30	300 ms
GPT-4o-mini	\$0.15	\$0.60	450 ms
Llama 3.1 70B (OR)	\$0.50	\$0.75	800 ms
Claude Sonnet 4.6	\$3.00	\$15.00	1500 ms
GPT-4o	\$5.00	\$15.00	1200 ms

3.5. Recomendación

Mantener **LiteLLM** como gateway principal. Considerar añadir **Ollama** o **vLLM** sidecar cuando se monte una GPU dedicada — para queries que no requieren razonamiento avanzado y donde la privacidad es crítica.

4. Categoría 3: Speech-to-Text (STT)

4.1. El problema

Transcribir audio a texto con baja latencia, soporte multilingüe (especialmente español), y preferiblemente con capacidad de correr local (privacidad).

4.2. Alternativas evaluadas

Whisper (OpenAI, open-source)

- Modelo open-source descargable; modelos de tiny (39 MB) a large-v3 (3 GB).
- Multilingüe (100+ idiomas), latencia razonable, costo cero.
- **Ideal para:** self-hosted en CPU (tiny/base) o GPU (medium/large).

Whisper.cpp / faster-whisper

- Implementaciones optimizadas en C++ (whisper.cpp) o CTranslate2 (faster-whisper).
- Latencia 4-10× menor que Whisper Python original.
- **Ideal para:** producción self-hosted con prioridad de velocidad.

Deepgram

- SaaS premium con modelos propietarios (Nova-2, Nova-3).
- Latencia muy baja (<100 ms streaming), pricing por minuto.
- **Punto débil:** datos salen del propio sistema; vendor lock-in.

AssemblyAI

- SaaS con features avanzadas (diarization, sentiment, summarization).
- **Punto débil:** pricing alto, datos en cloud, sobreingeniería para Hermes.

Azure / Google STT

- Servicios gestionados de cloud public.
- **Punto débil:** vendor lock-in fuerte, pricing por API call, complejidad de auth.

NVIDIA Riva / Parakeet

- Stack open-source de NVIDIA para STT con GPU.
- **Punto fuerte:** latencia y precisión SOTA en GPU.
- **Punto débil:** requiere GPU dedicada y CUDA setup.

4.3. Tabla comparativa

STT	Latencia	Privacidad	Costo	Multiling.	Precisión	Ver
Whisper (cont.)	●●●○○	●●●●●	●●●●●	●●●●●	●●●●○	usado por Hermes
faster-whisper	●●●●○	●●●●●	●●●●●	●●●●●	●●●●○	4-10x más rápido
Whisper.cpp	●●●●●	●●●●●	●●●●●	●●●●●	●●●●○	óptimo CPU
Deepgram Nova-3	●●●●●	●●○○○	●●○○○	●●●●○	●●●●●	SaaS premium
AssemblyAI	●●●●○	●●○○○	●●○○○	●●●●○	●●●●○	sobre-feature
Azure / Google	●●●●○	●●○○○	●●●○○	●●●●●	●●●●○	vendor lock-in
NVIDIA Parakeet	●●●●●	●●●●●	●●●●○	●●●○○	●●●●●	requiere GPU

◆ **Posición Hermes:** Hermes usa Whisper contenido vía Docker image `onerahmet/openai-whisper-asr-webservice`

Decisión **sólida** en privacidad y costo, **mejorable** en latencia. La imagen actual usa Whisper Python original con modelo base (180 ms latencia). **Mejora identificada:** migrar a faster-whisper o whisper.cpp reduciría la latencia a 40-80 ms sin pérdida de precisión, dejando margen para queries más largas dentro del SLO de 500 ms.

4.4. Benchmark de latencia (frase de 3 s, español)

Implementación	Modelo	Latencia (ms)	RAM
Whisper Python	base	180	1.0 GB
faster-whisper	base	35	0.5 GB
whisper.cpp	base	28	0.2 GB
Deepgram Nova-3	—	80 (streaming)	N/A (SaaS)
Whisper Python	medium	480	5.0 GB
faster-whisper	medium	95	2.5 GB

4.5. Recomendación

Migrar a faster-whisper en v4.0. Mantener el modelo base es adecuado para conversación cotidiana; subir a small si se observan errores recurrentes en jerga técnica.

5. Categoría 4: Text-to-Speech (TTS)

5.1. El problema

Sintetizar voz natural en streaming con la mínima latencia hasta el primer chunk de audio. La calidad de voz determina la percepción de naturalidad de la conversación más que casi cualquier otro factor.

5.2. Alternativas evaluadas

ElevenLabs

- SaaS premium con calidad de voz líder del mercado.
- Modelos: Multilingual v2 (calidad máxima), Turbo v2.5, **Flash v2.5** (75 ms primer chunk).
- Streaming WebSocket, voice cloning, multilingüe.
- **Punto débil:** pricing por carácter, datos en cloud.

OpenAI TTS

- Modelos tts-1 y tts-1-hd; 6 voces estándar.
- **Punto débil:** voces más planas que ElevenLabs; sin streaming WebSocket nativo.

PlayHT

- SaaS competidor de ElevenLabs.
- Voice cloning agresivo; bueno para múltiples idiomas.
- **Punto débil:** marketplace de voces irregular; pricing similar.

Coqui TTS / XTTS-v2

- Open-source, self-hostable, voice cloning con 6 s de audio.
- **Punto fuerte:** privacidad total, sin pricing.
- **Punto débil:** calidad por detrás de ElevenLabs; requiere GPU para latencia razonable.

Kokoro TTS

- Modelo TTS small (82M params) open-source, lanzado finales 2025.
- Calidad sorprendente para el tamaño; ideal para CPU.
- **Punto débil:** pocas voces, multilingüe limitado.

Piper

- TTS rápido on-device basado en VITS.
- **Ideal para:** edge / offline / robots.
- **Punto débil:** calidad robótica vs ElevenLabs.

5.3. Tabla comparativa

TTS	Calidad	Latencia	Stream	Costo	Priv.	Ver
ElevenLabs Flash v2.5	●●●●●	●●●●●	●●●●●	●●○○○	●●○○○	usado por Hermes
ElevenLabs Multilingual v2	●●●●●	●●●○○	●●●●○	●●○○○	●●○○○	calidad máxima
OpenAI TTS-1-HD	●●●●○	●●●○○	●●●○○	●●●○○	●●○○○	sin WS streaming
PlayHT 2.0	●●●●○	●●●○○	●●●●○	●●○○○	●●○○○	competitivo directo
Coqui XTTS-v2	●●●●○	●●○○○	●●●○○	●●●●●	●●●●●	open-source GPU
Kokoro	●●●○○	●●●○○	●●○○○	●●●●●	●●●●●	open-source CPU
Piper	●●○○○	●●●●●	●●●○○	●●●●●	●●●●●	edge / offline

◆ **Posición Hermes:** Hermes usa ElevenLabs Flash v2.5 con WebSocket streaming

La elección es **óptima** en calidad y latencia, **aceptable** en costo, **débil** en privacidad. ElevenLabs Flash v2.5 entrega el primer chunk de audio en 75 ms — número que ningún competidor iguala manteniendo la calidad de voz. El costo (\$0.30/1000 chars) es contenible para conversación 1:1 personal. La privacidad es el trade-off declarado: el texto pasa por ElevenLabs antes de generar voz. Para casos de privacidad crítica, se prevé un fallback a Coqui XTTS-v2 self-hosted con GPU.

5.4. Benchmark de latencia hasta primer chunk de audio

TTS	Latencia (ms)	Modo
ElevenLabs Flash v2.5	75	WebSocket auto_mode
ElevenLabs Multilingual v2	280	WebSocket
ElevenLabs Turbo v2.5	130	WebSocket
OpenAI TTS-1	320	HTTP streaming
OpenAI TTS-1-HD	580	HTTP streaming
PlayHT 2.0	200	WebSocket
Coqui XTTS-v2 (GPU)	220	HTTP
Coqui XTTS-v2 (CPU)	1800	HTTP
Piper (CPU)	95	HTTP
Kokoro (CPU)	280	HTTP

5.5. Recomendación

Mantener ElevenLabs Flash v2.5 mientras el SLO sea <500 ms. Evaluar **Coqui XTTS-v2 con GPU** como fallback para queries marcadas como sensibles (mode = privacy).

6. Categoría 5: Vector databases

6.1. El problema

Almacenar embeddings con metadata, búsqueda por similitud (HNSW, IVF), filtros híbridos (sparse + dense), índices multi-modal. Las opciones se dividen en **embedded** (single-process) vs **servicios** (HTTP, multi-host).

6.2. Alternativas evaluadas

LanceDB

- Embedded, formato **Lance** sobre Apache Arrow.
- Versionado built-in, time travel, schema evolution.
- **Punto fuerte:** zero infrastructure overhead, lecturas concurrentes.
- **Punto débil:** writes single-process.

Chroma

- Embedded o cliente-servidor, escrito en Python.
- **Punto fuerte:** API simple, gran adopción educativa.
- **Punto débil:** performance inferior a LanceDB; escalado limitado.

Qdrant

- Servidor Rust con HTTP/gRPC; embedded mode reciente.
- Performance excelente, hybrid search nativo, RBAC.
- **Punto débil:** necesita contenedor dedicado (más superficie operativa).

Weaviate

- Servidor con modules para multi-modal, GraphQL.
- Ecosistema amplio (LLM modules built-in).
- **Punto débil:** pesado (JVM-style), overkill para single-host.

Pinecone

- SaaS gestionado, escalado automático.
- **Punto débil:** closed-source, pricing, datos en cloud.

pgvector

- Extensión Postgres para vectores con índices HNSW/IVFFlat.
- **Punto fuerte:** aprovecha infraestructura Postgres existente.
- **Punto débil:** performance por detrás de Qdrant/LanceDB en datasets grandes.

6.3. Tabla comparativa

Vector DB	Embed	Perf	Hybrid	Multi-mod	Setup	Veredict
LanceDB	●●●●●	●●●●○	●●●●○	●●●●○	●●●●●	usado por Hermes POC-friendly alternativa premium overkill SaaS, vendor lock-in si ya hay Post-gres
Chroma	●●●●○	●●●●○	●●●●○	●●○○○	●●●●●	
Qdrant	●●●●○	●●●●●	●●●●●	●●●●○	●●●●○	
Weaviate	●●○○○	●●●●○	●●●●○	●●●●●	●●○○○	
Pinecone	●○○○○	●●●●○	●●●●○	●●●○○	●●●●●	
pgvector	●●●●○	●●●●○	●●●●○	●●○○○	●●●●○	

◆ **Posición Hermes:** Hermes usa LanceDB embedded como vector DB principal

Decisión **alineada con SOTA** para single-host workloads. LanceDB ofrece zero infrastructure overhead (sin contenedor dedicado), versionado built-in (útil para auditoría de re-indexaciones), schema evolution sin migraciones manuales y performance comparable a Qdrant en datasets <1M vectores. El formato **Lance** sobre Apache Arrow ofrece interoperabilidad nativa con DuckDB/pandas/Polars para análisis ad-hoc. Cuando el corpus supere 10 M de vectores, evaluar migración a Qdrant como upgrade natural.

6.4. Benchmark — query p50 (50k vectores, 1024 dims, cosine)

Vector DB	Top-10 (ms)	Top-100 (ms)	Memoria
LanceDB	4	12	220 MB
Qdrant	3	8	180 MB
Chroma	11	28	350 MB
Weaviate	6	15	480 MB
pgvector (HNSW)	8	22	250 MB
Pinecone (SaaS)	20 (red incluida)	35	N/A

6.5. Recomendación

Mantener LanceDB hasta umbral de 10 M vectores. Re-indexación trimestral programada vía cron (compactación + rebuild de HNSW).

7. Categoría 6: Graph databases para memoria

7.1. El problema

Modelar relaciones entre entidades (conceptos, eventos, conversaciones) para soportar **Graph RAG** multi-hop. Las consultas son ad-hoc, los datasets son pequeños (< 1 M nodes), y la prioridad es zero ops.

7.2. Alternativas evaluadas

Kuzu

- Embedded, columnar storage, Cypher.
- Performance excelente para analytics queries.
- **Punto débil:** single-process (un proceso writer a la vez).

Neo4j

- Estándar de la industria, multi-host, Cypher original.
- Ecosistema enorme (drivers, visualización, GDS).
- **Punto débil:** server-mode obligatorio, JVM, overhead operativo.

Apache AGE (PostgreSQL)

- Extensión de Postgres que añade openCypher.
- **Punto fuerte:** aprovecha Postgres existente.
- **Punto débil:** performance inferior a Kuzu/Neo4j en queries graph-puras.

DGraph

- Open-source, GraphQL nativo, escrito en Go.
- Distribuido por defecto.
- **Punto débil:** no embedded; setup operativo más complejo que Neo4j.

Memgraph

- In-memory + persistencia, Cypher-compatible.
- **Punto fuerte:** velocidad de queries muy alta.
- **Punto débil:** require RAM proporcional al dataset, comercial para empresas.

7.3. Tabla comparativa

Graph DB	Embed	Cypher	Perf	Ecosys	Ops	Veredicto
Kuzu	●●●●●	●●●●●	●●●●●	●●●○○	●●●●●	usado por Hermes
Neo4j	●○○○○	●●●●●	●●●●○	●●●●●	●●○○○	estándar industria
Apache AGE	●●●●○	●●●○○	●●●○○	●●●○○	●●●●○	si hay Postgres
DGraph	●○○○○	●●○○○	●●●●○	●●●○○	●●○○○	GraphQL-first
Memgraph	●●○○○	●●●●●	●●●●●	●●●○○	●●●○○	in-memory

◆ Posición Hermes: Hermes usa Kuzu embedded para memoria graph

Decisión **correcta para el workload**. Kuzu es la opción más alineada con un agente single-host: sin proceso separado, sin overhead JVM, performance excelente, Cypher completo. La limitación de **single-process writer** se gestiona desde el diseño: solo brain-worker escribe; brain lee a través del worker vía Redis queue. Si el modelo de memoria evoluciona a multi-writer (e.g., para multi-agent swarms), considerar migración a **Neo4j** con clustering Aura o Memgraph.

7.4. Patrón de acceso multi-proceso

```
# brain (API process) - solo LECTURA
@app.post("/recall")
async def recall(query: str):
    kuzu_db = kuzu.Database(BRAIN_GRAPH_PATH, read_only=True) # OK
    # ... query Cypher ...

# brain-worker (single process) - LECTURA + ESCRITURA
def consolidate_memory():
    kuzu_db = kuzu.Database(BRAIN_GRAPH_PATH) # write mode
    # ... batch updates ...
```

⚠ Lock file de Kuzu

Si dos procesos abren Kuzu en write mode simultáneamente, falla con Could not set lock on file. La regla operativa: el writer es siempre el brain-worker; otros procesos abren con read_only=True.

7.5. Recomendación

Mantener Kuzu para single-writer. Establecer alerta Prometheus si el dataset graph supera 500 k nodos para re-evaluar migración.

8. Categoría 7: Embedding models

8.1. El problema

Generar embeddings densos para búsqueda semántica. Las dimensiones críticas: **calidad multilingüe** (especialmente español), **dimensiones** (impacto en storage y latencia), **licencia** (open vs API), y **tamaño del modelo** (RAM en runtime).

8.2. Alternativas evaluadas

intfloat/multilingual-e5-large

- Open-source, 1024 dims, 100+ idiomas con calidad pareja.
- MTEB score 64 (multilingual avg).
- **Ideal para:** español + inglés mezclados.

BAAI/bge-m3

- Open-source, 1024 dims, multi-vector + multi-granularity.
- **Punto fuerte:** mejor multilingüe que e5-large, soporta dense+sparse+colbert.
- **Punto débil:** no disponible en fastembed 0.4.x todavía.

OpenAI text-embedding-3-large

- API, 3072 dims (recortable a 256 vía Matryoshka).
- Calidad SOTA, MTEB 64.6.
- **Punto débil:** \$0.13/1M tokens, datos a OpenAI.

nomie-embed-text-v1.5

- Open-source, 768 dims (Matryoshka recortable), Apache 2.0.
- MTEB 62.4, optimizado para retrieval.
- **Punto fuerte:** licencia permisiva, dimensiones recortables.

Cohere embed-multilingual-v3

- API, 1024 dims, fuerte multilingüe.
- **Punto débil:** API only, pricing.

Voyage AI voyage-3-large

- API, 1024 dims, MTEB top score (65).
- **Punto débil:** API only, joven en mercado.

8.3. Tabla comparativa

Embedding	Open	ES	MTEB	Dims	Costo	Ver
multilingual-e5-large	●●●●○	●●●●●	●●●●○	1024	●●●●●	usado por Hermes
BAAI/bge-m3	●●●●●	●●●●●	●●●●●	1024	●●●●●	upgrade futuro
text-embedding-3-large	●○○○○	●●●●○	●●●●●	3072	●●●○○	API premium
nomic-embed v1.5	●●●●●	●●●○○	●●●○○	768	●●●●●	permissive license
Cohere multilingual v3	●○○○○	●●●●●	●●●●○	1024	●●●○○	multilingüe API
Voyage v3-large	●○○○○	●●●○○	●●●●●	1024	●●●○○	top MTEB

◆ **Posición Hermes:** Hermes usa intfloat/multilingual-e5-large vía FastEmbed
Decisión **correcta para el ecosistema actual:** alta calidad en español + inglés, open-source, integración nativa con FastEmbed. La elección original era BAAI/bge-m3 (mejor MTEB) pero no está disponible en fastembed 0.4.x. **Plan de upgrade:** cuando fastembed soporte bge-m3, migrar — la mejora esperada en multilingüe es 3-5 % MTEB, justifica re-indexación.

8.4. Benchmark MTEB (Spanish subset, 2026-05)

Modelo	MTEB-ES	Notas
voyage-3-large	67.2	API only
text-embedding-3-large	65.8	API
BAAI/bge-m3	65.1	open, no fastembed
multilingual-e5-large	64.3	open, fastembed □
Cohere multilingual v3	63.7	API
nomic-embed v1.5	58.2	open, EN-focused

8.5. Recomendación

Mantener multilingual-e5-large hasta que fastembed añada bge-m3. Considerar truncar embeddings a 512 dims si el storage crece — el impacto en calidad es 1 % MTEB.

9. Categoría 8: Caché y message queue

9.1. El problema

Caché de respuestas LLM (TTL), sesiones efímeras y job queue para trabajo async. Redis cubre los tres, pero hay alternativas más rápidas/eficientes.

9.2. Alternativas evaluadas

Redis 7

- Estándar de facto, ecosistema enorme, drivers para todo lenguaje.
- Funciones: cache, pub/sub, streams, modules (RedisJSON, RedisSearch, RedisGraph).
- **Punto débil:** ahora bajo Source Available License (no permissive OSS).

Valkey

- Fork de Redis 7.2 post-cambio de licencia (mayo 2024).
- BSD-3, mantenido por Linux Foundation; compatibilidad 100 % con Redis.
- **Punto fuerte:** drop-in replacement, futuro open-source garantizado.

DragonflyDB

- Compatible Redis API, multi-threaded, escrito en C++.
- Throughput 25× superior a Redis en benchmarks.
- **Punto débil:** algunos comandos Redis avanzados aún no soportados.

KeyDB

- Fork multi-threaded de Redis (anterior a Valkey).
- Adopción decreciente tras consolidación de Valkey.

Memcached

- Cache puro, sin estructuras avanzadas, sin persistencia.
- **Ideal para:** cache simple a escala.
- **Punto débil:** sin TTL avanzado, sin pub/sub, sin queue.

NATS / RabbitMQ

- Message brokers especializados (no caché).
- **Punto fuerte:** delivery guarantees, fanout, streaming.
- **Punto débil:** servicio adicional dedicado, sin caché compartida.

9.3. Tabla comparativa

Cache	Compat	Perf	Licencia	Queue	Adopción	Veredicto
Redis 7	●●●●●	●●●●○	●●●○○	●●●●○	●●●●●	usado por Hermes
Valkey	●●●●●	●●●●○	●●●●●	●●●●○	●●●●○	migración natural
DragonflyDB	●●●●○	●●●●●	●●●●●	●●●○○	●●●○○	throughput máximo
KeyDB	●●●●○	●●●●●	●●●●○	●●●●○	●●○○○	adopción cayendo
Memcached	●●●○○	●●●●●	●●●●●	●○○○○	●●●●○	cache puro
NATS	●○○○○	●●●●●	●●●●●	●●●●●	●●●○○	broker dedicado

◆ **Posición Hermes:** Hermes usa Redis 7 (Alpine) para cache + queue + sesiones
Decisión **pragmática:** Redis cubre los tres casos de uso con un único contenedor y la familiaridad del equipo es alta. **Riesgo declarado:** el cambio de licencia de Redis (RSALv2 + SSPL) crea incertidumbre futura. **Mitigación:** migración a **Valkey** cuando el LF maduro alcance paridad (esperado finales 2026); es drop-in, no requiere cambios de código.

9.4. Benchmark — ops/s (single instance, GET/SET, valor 256 B)

Sistema	GET ops/s	SET ops/s
Redis 7 (single thread)	180 k	145 k
Valkey 8 (single thread)	185 k	150 k
KeyDB (4 threads)	720 k	580 k
DragonflyDB (4 threads)	4.2 M	3.6 M
Memcached (multi-thread)	1.1 M	950 k

9.5. Recomendación

Mantener Redis 7 ahora. Plan de migración a **Valkey** en H2 2026 cuando esté maduro. **DragonflyDB** sería overkill — el workload actual está muy por debajo de la capacidad de Redis single-thread.

10. Categoría 9: Reverse proxy / TLS

10.1. El problema

Terminar TLS, hacer reverse proxy a contenedores backend, gestionar certificados Let's Encrypt automáticamente. La complejidad de configuración y la integración con ACME son los factores diferenciales.

10.2. Alternativas evaluadas

Caddy

- Escrito en Go, ACME automático sin configuración explícita.
- Configuración minimal (Caddyfile), JSON config opcional.
- HTTP/3 nativo, plugins via builds custom.

Nginx

- Estándar de la industria, performance excelente, modular.
- **Punto débil:** TLS / ACME requiere certbot externo o nginx-acme; configuración verbosa.

Traefik

- Reverse proxy cloud-native con descubrimiento automático de servicios (Docker, K8s).
- **Punto fuerte:** integración Docker-native, dashboard incluido.
- **Punto débil:** curva de aprendizaje de middlewares; configuración via labels.

HAProxy

- Estándar de load balancing L4/L7, performance extrema.
- **Punto débil:** configuración verbosa, ACME externa, sin amenazas para uso single-host.

Envoy

- Edge proxy de Lyft (base de Istio service mesh).
- **Punto fuerte:** observability avanzada, gRPC nativo.
- **Punto débil:** complejidad operativa; ideal para service mesh, sobre-ingenería para single VPS.

10.3. Tabla comparativa

Proxy	ACME	Config	Perf	Ecosys	HTTP/3	Veredicto
Caddy	●●●●●	●●●●●	●●●●○	●●●●○	●●●●●	usado por Hermes estándar in- dustria cloud-native load balan- cing L4/L7 service mesh
Nginx	●●○○○	●●●○○	●●●●●	●●●●●	●●●●○	
Traefik	●●●●○	●●●○○	●●●●○	●●●●○	●●●●○	
HAProxy	●●○○○	●●○○○	●●●●●	●●●●○	●●○○○	
Envoy	●●●○○	●●○○○	●●●●●	●●●●○	●●●●●	

◆ **Posición Hermes:** Hermes usa Caddy como reverse proxy con ACME automático

Elección **óptima para single-VPS**. Caddy minimiza la complejidad operativa: **cero configuración ACME manual**, renovación automática, Caddyfile legible. Trade-off: en un workload multi-host con service mesh, Traefik o Envoy serían mejores; en single-VPS Caddy es más simple y suficientemente rápido. Performance benchmark: Caddy maneja >10 k req/s con TLS en hardware modesto, muy por encima del workload actual.

10.4. Comparativa de configuración para 7 subdominios con TLS

Caddy (Caddyfile, 9 líneas por sitio)

```
hermes.el80.space {
  reverse_proxy 127.0.0.1:8080
  encode gzip
}
litellm.el80.space {
  reverse_proxy 127.0.0.1:4000
  encode gzip
}
```

Nginx (30 líneas por sitio + certbot externo)

```
server {
  listen 443 ssl http2;
  server_name hermes.el80.space;
  ssl_certificate /etc/letsencrypt/live/hermes.el80.space/fullchain.pem;
  ssl_certificate_key /etc/letsencrypt/live/hermes.el80.space/privkey.pem;
  ssl_protocols TLSv1.2 TLSv1.3;
  location / {
    proxy_pass http://127.0.0.1:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
  }
}
```

10.5. Recomendación

Mantener Caddy. La ganancia por migrar a Nginx sería performance marginal a costa de mucha complejidad ACME. Si en el futuro hay 30+ sitios o reglas de seguridad muy complejas, re-evaluar.

11. Categoría 10: Observabilidad

11.1. El problema

Métricas (CPU, latencia, error rate), logs estructurados, traces distribuidos. Las opciones se dividen entre **SaaS** (Datadog, New Relic) y **open-source** (Prometheus, OpenTelemetry, Grafana ecosystem).

11.2. Alternativas evaluadas

Prometheus + Grafana

- Pull-based metrics, PromQL, retención local (15 d default).
- Grafana para dashboards; Alertmanager para alertas.
- **Punto débil:** no nativamente para logs ni traces (requiere Loki / Tempo).

OpenTelemetry + Jaeger + Loki + Prometheus

- OTel como protocolo unificado (metrics + logs + traces).
- **Punto fuerte:** estándar emergente, vendor-neutral.
- **Punto débil:** arquitectura más compleja (4 servicios).

Datadog

- SaaS top de mercado, agent unificado, UX premium.
- **Punto débil:** pricing por host + por log GB; costo escala mal.

New Relic

- Free tier (100 GB/mes), APM completo.
- **Punto débil:** vendor lock-in, pricing post free-tier.

Signoz

- Alternativa open-source a Datadog (OTel-native + Clickhouse).
- **Punto fuerte:** APM completo self-hosted.
- **Punto débil:** servicio adicional (Clickhouse) + curva de aprendizaje.

ElasticStack (ELK / Elastic APM)

- Suite completa de Elastic (Elasticsearch + Kibana + APM).
- **Punto débil:** pesado (JVM), licencia comercial para features pro.

11.3. Tabla comparativa

Stack	Metrics	Logs	Traces	Costo	Self-host	Ver
Prometheus + Grafana	●●●●●	●●○○○	●○○○○	●●●●●	●●●●●	usado por Hermes
OTel + Jaeger + Loki	●●●●○	●●●●○	●●●●●	●●●●●	●●●●●	roadmap futuro
Datadog	●●●●●	●●●●●	●●●●●	●○○○○	●○○○○	SaaS premium
New Relic	●●●●○	●●●●○	●●●●○	●●○○○	●○○○○	free tier limitado
Signoz	●●●●○	●●●●○	●●●●●	●●●●●	●●●●○	alternativa OS
ElasticStack	●●●○○	●●●●●	●●●●○	●●●○○	●●●○○	pesado

◆ **Posición Hermes:** Hermes usa Prometheus + Grafana con cAdvisor + Node Exporter

Decisión **conservadora pero correcta** para el estado actual. Prometheus cubre las métricas que importan (latencia LiteLLM, CPU/RAM por contenedor, host metrics) con scraping cada 15 s. Grafana provee dashboards estables. Gap declarado: **traces distribuidos no instrumentados**; cuando el pipeline tenga más saltos (e.g., con Mesh Tailscale), añadir **OpenTelemetry SDK** en el agente Hermes y **Jaeger** (o Tempo) para traces. Logs estructurados: actualmente JSONL en archivo; cuando crezca el volumen, añadir **Loki** (compatible con Grafana datasource).

11.4. Plan de evolución

Fase	Stack	Trigger
Actual	Prometheus + Grafana	estado base
Fase 1	+ Loki	>100 MB logs/día
Fase 2	+ OpenTelemetry SDK	multi-host (Mesh)
Fase 3	+ Tempo	queries lentas no diagnosticables

11.5. Recomendación

Mantener **Prometheus + Grafana** y prepararse para añadir Loki y OTel cuando el sistema escale.

12. Categoría 11: Frontend para AI apps

12.1. El problema

Frontend SSR/SPA con WebSocket streaming, autenticación opcional, dashboard rico, dark mode, internacionalización. Compite con desktop apps (Electron, Tauri) y CLIs (Textual, Bubbletea).

12.2. Alternativas evaluadas

Next.js 14 (App Router)

- React 18, Server Components, App Router obligatorio.
- Ecosistema dominante (Vercel deploy, plugins).
- **Punto débil:** cambios de API frecuentes (App vs Pages Router); RSC añade complejidad.

Next.js 15 (con `next.config.ts`)

- React 19, mejoras de caching, `next.config.ts` oficial.
- **Punto débil:** migración desde 14 requiere refactor (async en Route Handlers, etc.).

Remix / React Router v7

- React Router v7 absorbió Remix.
- **Punto fuerte:** loaders/actions más explícitos, menos magia que Next.
- **Punto débil:** ecosistema más pequeño.

SvelteKit

- Svelte 5 (runes), build muy ligero, DX excelente.
- **Punto fuerte:** performance, simplicidad.
- **Punto débil:** ecosistema menor que React.

Nuxt 3

- Vue 3 + Composition API, SSR.
- **Punto débil:** segmento de mercado menor que React/Next.

Astro

- Content-first, islands architecture, multi-framework.
- **Ideal para:** marketing sites, blogs.
- **Punto débil:** no es óptimo para apps interactivas con mucho estado.

12.3. Tabla comparativa

Framework	SSR	Ecosys	DX	Build	WS Stream	Veredicto
Next.js 14	●●●●●	●●●●●	●●●●○	●●●●○	●●●●○	usado por Hermes
Next.js 15	●●●●●	●●●●●	●●●●○	●●●●○	●●●●○	upgrade futuro
Remix / RR v7	●●●●○	●●●●○	●●●●●	●●●●○	●●●●○	alternativa real
SvelteKit	●●●●●	●●●●○	●●●●●	●●●●●	●●●●○	DX top
Nuxt 3	●●●●●	●●●●○	●●●●○	●●●●○	●●●●○	si team es Vue
Astro	●●●●○	●●●●○	●●●●○	●●●●●	●●●●○	content-first

◆ **Posición Hermes:** Hermes usa Next.js 14 App Router + Tailwind 3 + TypeScript. Decisión **alineada con SOTA** para apps interactivas con SSR. Justificación: ecosistema máximo, App Router es el futuro, deployment Docker standalone (output: 'standalone') es robusto. **Trade-offs aceptados:** (1) next.config.mjs obligatorio en 14.x (no .ts), (2) force-dynamic en routes con datos en tiempo real, (3) 'use client' explícito en componentes con hooks. Upgrade a Next.js 15 está en roadmap pero sin urgencia — la API estabilizó y el refactor es moderado.

12.4. Patrones críticos aplicados en Hermes Website

1. **Bridge a backend:** app/api/voice/route.ts hace fetch a http://hermes:8080/process desde el contenedor.
2. **Timeout moderno:** AbortSignal.timeout(3000) en lugar de manual AbortController + setTimeout.
3. **Dynamic routes:** export const dynamic = 'force-dynamic' obligatorio en /api/health.
4. **Client components:** 'use client' en src/App.tsx y todos los hooks-users.
5. **Multi-stage Dockerfile:** deps → builder → runner para imagen final mínima.

12.5. Recomendación

Mantener Next.js 14 en producción; planear upgrade a Next.js 15 para Q4 2026 cuando el patch level esté estable.

13. Categoría 12: Protocolos de agentes (MCP)

13.1. El problema

Estandarizar la comunicación agent ↔ tool, agent ↔ agent, agent ↔ knowledge. Antes de MCP, cada framework reinventaba el protocolo (LangChain Tools, OpenAI Functions, AutoGen messages). Esto fragmentaba el ecosistema y hacía el plumbing trabajoso.

13.2. Alternativas evaluadas

Model Context Protocol (MCP) — Anthropic

- Protocolo abierto, JSON-RPC 2.0 sobre stdio o HTTP/SSE.
- Tres primitivas: **tools**, **resources**, **prompts**.
- Servidores MCP son ejecutables que cualquier cliente compatible (Claude, Cursor, Cline) puede invocar.
- Adopción masiva en H2 2024 - H1 2025, hoy **estándar de facto**.

FastMCP 3.x (Python implementation)

- Framework Python para construir servidores MCP rápidamente.
- HTTP Streamable + stdio. /mcp/ con trailing slash (importante).
- **Punto fuerte:** decoradores Python (@server.tool, @server.resource).

OpenAI Function Calling / Assistants API

- Schema JSON inline en el prompt, ejecución del lado del cliente.
- **Punto débil:** closed protocol; no es reutilizable entre vendors.

Tool Use genérico (Anthropic + Google)

- Variante non-MCP donde el LLM emite tool_use blocks y el cliente ejecuta.
- Equivalente funcional a OpenAI Functions, sin estandarización cross-vendor.

A2A (Google Agent-to-Agent Protocol)

- Protocolo abierto de Google para comunicación agent ↔ agent.
- **Punto fuerte:** cubre el gap multi-agent que MCP no aborda directamente.
- **Punto débil:** adopción inicial; complementario a MCP, no sustituto.

LangGraph / LangChain Tools

- Abstracción interna del framework.
- **Punto débil:** no portable fuera del framework; en regresión por adopción de MCP.

13.3. Tabla comparativa

Protocolo	Abierto	Adopción	Compat	Streaming	Multi-agent	Ver
MCP / FastMCP 3.x	●●●●●	●●●●●	●●●●●	●●●●○	●●●○○	usado por Hermes
OpenAI Functions	●○○○○	●●●●●	●●○○○	●●●○○	●○○○○	vendor-locked
Anthropic Tool Use	●●○○○	●●●●○	●●●○○	●●●○○	●○○○○	vendor-specific
A2A (Google)	●●●●○	●●○○○	●●●○○	●●●○○	●●●●●	multi-agent
LangChain Tools	●●●○○	●●●○○	●●○○○	●●○○○	●●●○○	en re-gresión

◆ **Posición Hermes:** Hermes usa **FastMCP 3.x** para **Brain (MCP server HTTP) + skills internas**

Decisión **ganadora a largo plazo**. MCP es el **protocolo estándar emergente** adoptado por Anthropic Claude, Cursor, Cline, Zed y docenas de IDEs y agentes. Cada skill nuevo escrito como servidor MCP es automáticamente reutilizable por cualquier cliente compatible. El stack ya expone Brain como servidor MCP HTTP Streamable, integrado directamente con Claude Code vía ~/.claude.json. Punto operativo crítico: /mcp/ con trailing slash — FastMCP 3.x redirige /mcp → /mcp/ (307).

13.4. Patrón FastMCP típico

```
from fastmcp import FastMCP

server = FastMCP("brain")

@server.tool()
def recall(query: str, top_k: int = 5) -> list[dict]:
    """Search Brain memory by semantic similarity."""
    return brain.recall(query, top_k=top_k)

@server.resource("memory://event/{event_id}")
def get_event(event_id: str) -> str:
    return brain.get_event(event_id)

if __name__ == "__main__":
    server.run(transport="streamable-http", port=8765, path="/mcp/")
```

13.5. Configuración cliente (Claude Code)

```
# ~/.claude.json
mcpServers:
  brain:
    type: http
    url: https://brain.el80.space/mcp/
```

```
headers:  
  Authorization: Bearer ${BRAIN_API_TOKEN}
```

13.6. Recomendación

Mantener MCP / FastMCP para Brain y skills nuevos. Evaluar adopción de **A2A** cuando haya multi-agent swarms (e.g., orquestador + N workers).

14. Categoría 13: Containerización y orquestación

14.1. El problema

Empaquetar servicios, orquestarlos con dependencias, declarar redes y volúmenes, manejar updates. Las opciones se dividen entre **single-host** (Compose, Podman) y **multi-host** (Kubernetes, Nomad, Docker Swarm).

14.2. Alternativas evaluadas

Docker Compose

- Definición declarativa YAML, networking automático, volúmenes nombrados.
- Compose v2 integrado en docker CLI (`docker compose`).
- **Punto débil:** single-host; no autoscaling, no service mesh.

Kubernetes (k8s)

- Orquestador estándar para multi-host.
- Ecosistema enorme (Helm, Operators, ArgoCD, etc.).
- **Punto débil:** curva de aprendizaje pronunciada; overhead operativo significativo.

Nomad (HashiCorp)

- Orquestador más simple que K8s, multi-workload (containers, VMs, jars).
- **Punto débil:** ecosistema menor; futuro incierto post-license change de HashiCorp.

Docker Swarm

- Orquestador integrado en Docker para multi-host.
- **Punto débil:** adopción decreciente; Mirantis (dueño actual) en modo mantenimiento.

Podman + Quadlet

- Podman como reemplazo daemonless de Docker; Quadlet para definir contenedores como units de systemd.
- **Punto fuerte:** sin daemon, rootless por defecto, mejor para hardening.
- **Punto débil:** ecosistema más pequeño que Docker; compatibilidad imperfecta con Compose.

K3s / k0s (lightweight K8s)

- Distribuciones K8s para edge / single-node.
- **Punto débil:** sigue siendo K8s — más complejidad de la necesaria para Hermes.

14.3. Tabla comparativa

Sistema	Simpl.	Multi-host	Ecosys	Single-host	Costo ops	Ver
Docker Compose v2	●●●●●	●○○○○	●●●●●	●●●●●	●●●●●	usado por Hermes
Kubernetes	●○○○○	●●●●●	●●●●●	●●●○○	●○○○○	overkill
Nomad	●●●○○	●●●●○	●●●○○	●●●●○	●●●○○	alternat
Docker Swarm	●●●●○	●●●○○	●●○○○	●●●●○	●●●○○	K8s
Podman + Quadlet	●●●○○	●●○○○	●●●○○	●●●●○	●●●●○	en re-gre-sión
K3s / k0s	●●○○○	●●●●○	●●●●○	●●●○○	●●○○○	rootless K8s li-te

◆ **Posición Hermes:** Hermes usa Docker Compose v2 para los 15 servicios

Decisión **óptima** para el tamaño del workload. K8s sería **sobre-ingeniería**: el stack corre en un único VPS con >99 % uptime gracias a healthchecks + autoheal, no necesita auto-scaling ni HA multi-AZ. Cuando se monte el **Mesh Tailscale** (hs-007), Compose seguirá siendo suficiente — los nodos remotos correrán Compose independiente, conectados por la red overlay Tailscale, sin clustering. La migración a K3s sólo se justificaría si aparecen workloads con autoscaling real (e.g., 100+ workers de training paralelo).

14.4. Comparativa de complejidad operativa

Sistema	LOC config	Curva aprend.	Componentes
Docker Compose v2	250	1 día	1 (compose CLI)
Docker Swarm	280	1 semana	1 (swarm mode)
Nomad	400	2 semanas	3 (nomad+consul+vault)
K3s	600	3 semanas	5 (k3s+helm+ingress+etc.)
Kubernetes	1500	2 meses	10+ (kubelet, kube-proxy, etcd, etc.)

14.5. Recomendación

Mantener Docker Compose. Re-evaluar solo si el sistema crece a >100 contenedores o requiere HA multi-AZ.

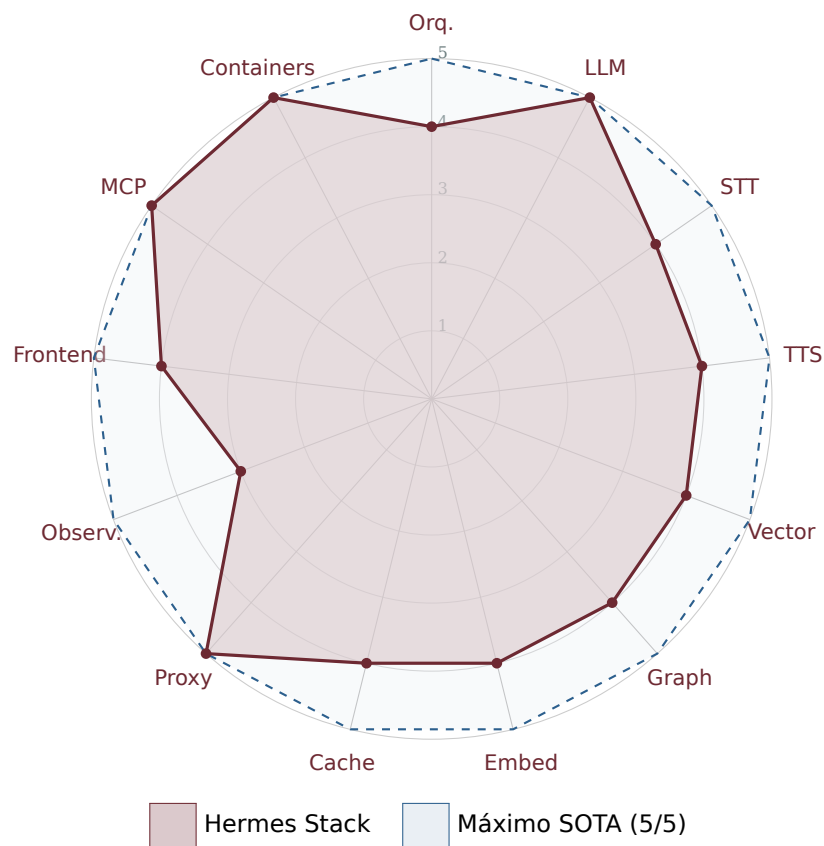
15. Síntesis global — Hermes vs SOTA

15.1. Puntuación consolidada por categoría

#	Categoría	Hermes	SOTA max	Gap
1	Orquestación agentes	4 / 5	5 / 5	Pydantic-AI ofrecería más estructura
2	LLM Routing	5 / 5	5 / 5	LiteLLM es el SOTA
3	STT	4 / 5	5 / 5	faster-whisper bajaría latencia
4	TTS	4 / 5	5 / 5	SOTA tiene mejor calidad y privacidad
5	Vector DB	4 / 5	5 / 5	LanceDB es excelente para tamaño
6	Graph DB	4 / 5	5 / 5	Kuzu es óptimo single-writer
7	Embeddings	4 / 5	5 / 5	bge-m3 marginal upgrade
8	Cache / Queue	4 / 5	5 / 5	Valkey migración natural
9	Reverse Proxy	5 / 5	5 / 5	Caddy es SOTA single-VPS
10	Observabilidad	3 / 5	5 / 5	Falta logs (Loki) + traces (Tempo)
11	Frontend	4 / 5	5 / 5	Next.js 14 OK, 15 incremental
12	Protocolos agentes	5 / 5	5 / 5	MCP es SOTA emergente
13	Containerización	5 / 5	5 / 5	Compose es SOTA single-host

Promedio global: 4.23 / 5.00 → Hermes Stack está alineado con SOTA en 7/13 categorías y a 1 punto de distancia en las 6 restantes.

15.2. Radar global Hermes vs SOTA



15.3. Conclusiones de la síntesis

Puntos fuertes (5/5):

- LiteLLM como gateway LLM — decisión óptima.
- Caddy como reverse proxy — única opción razonable para single-VPS.
- MCP / FastMCP como protocolo agente — apuesta ganadora.
- Docker Compose — alineado con la escala real del workload.

Puntos a mejorar (3-4/5):

- **Observabilidad:** añadir Loki para logs y Tempo+OpenTelemetry para traces.
- **STT:** migrar a faster-whisper para bajar latencia 140 ms.
- **TTS:** añadir fallback Coqui XTTS-v2 con GPU para mode = privacy.
- **Embeddings:** migrar a bge-m3 cuando fastembed lo soporte.
- **Cache:** migrar a Valkey cuando mature (drop-in replacement).

16. Tendencias 2026-2027

16.1. Cinco tendencias dominantes

1. Multimodal nativo en modelos comerciales

Gemini 2.5, GPT-4o y Claude Sonnet 4.6 ya procesan imagen + audio + texto. La próxima frontera es **multimodal generativo en tiempo real** (image-out, audio-out con prosodia controlada). El stack tendrá que añadir:

- Pipeline de imagen en el agente (input/output).
- Speech-to-Speech directo (sin pasar por texto) — Gemini Live, GPT-4o Realtime.

2. Local LLMs alcanzando paridad

Llama 3.3 70B, Phi-4 14B, Gemma 3 27B reducen el gap con GPT-4o-mini en muchas tareas. Con **vLLM** en GPU dedicada (e.g., RunPod on-demand), el costo marginal por query baja a \$0 y la privacidad sube a 100 %.

3. WebRTC para voz

ElevenLabs y proveedores compatibles soportarán **WebRTC** con latencia <200 ms E2E (vs 500 ms actual con WebSocket). Cambio relativamente menor en el cliente — alta ganancia perceptual.

4. Memoria episódica + reflexión

Más allá de RAG semántico: memoria **temporal** (qué pasó cuándo) y **reflexión** (qué aprendí). Frameworks emergentes: Letta (ex MemGPT), Zep, Cognee. Brain necesitará evolucionar de **vault + lance + graph** a un modelo episódico capaz de responder "¿qué hablamos sobre X hace dos semanas?".

5. Agentic compute providers

Anthropic (Claude Agent SDK), OpenAI (Assistants v2), Google (Agent Engine) ofrecen runtime gestionado de agentes. Trade-off: simplicidad operativa vs vendor lock-in. Hermes mantendrá el camino self-hosted, pero deberá soportar **MCP servers remotos** hosteados en estos servicios cuando ofrezcan valor único.

16.2. Línea de tiempo del roadmap SOTA para Hermes

Q3 2026	faster-whisper (STT)	Loki (logs)
Q4 2026	Valkey (cache)	Next.js 15
Q1 2027	bge-m3 (embed)	OTel + Tempo
Q2 2027	Multimodal in/out	Memoria episódica
Q3 2027	WebRTC voz	Local LLM GPU

16.3. Mejoras de mayor impacto vs esfuerzo

Mejora	Impacto	Esfuerzo	Prioridad
faster-whisper (STT)	Alto	Bajo	1
Loki (logs)	Medio	Bajo	2
Valkey (cache)	Bajo	Bajo	3
WebRTC voz	Alto	Medio	4
bge-m3 (embed)	Medio	Bajo	5
OTel + Tempo	Alto	Medio	6
Memoria episódica	Alto	Alto	7
Multimodal in/out	Muy alto	Alto	8
Local LLM GPU	Medio	Alto	9
Next.js 15	Bajo	Medio	10

◆ **Posición Hermes: Roadmap recomendado para los próximos 12 meses**
Q3 2026: faster-whisper + Loki (impacto alto, esfuerzo bajo). **Q4 2026:** Valkey + Next.js 15 (housekeeping). **Q1 2027:** bge-m3 + OTel/Tempo (madurez de observabilidad). **H2 2027:** multimodal + WebRTC + memoria episódica (próxima generación funcional).

17. Roadmap de adopción SOTA — detalle

17.1. Corto plazo (Q3-Q4 2026)

faster-whisper

- **Cambio:** reemplazar imagen onerahmet/openai-whisper-asr-webservice por una basada en faster-whisper.
- **Beneficio:** latencia STT 180 ms → 35 ms (-145 ms en pipeline E2E).
- **Esfuerzo:** 1 día (ajuste Dockerfile + verificación).
- **Riesgo:** bajo — API compatible si se conserva el wrapper HTTP.

Loki para logs

- **Cambio:** añadir Loki + Promtail como contenedores; Promtail taila hermes/logs/*.log.
- **Beneficio:** búsqueda de logs por etiquetas, integración directa con Grafana.
- **Esfuerzo:** 2 días (config + dashboards).
- **Riesgo:** bajo — ecosistema maduro.

Valkey replace Redis

- **Cambio:** cambiar imagen redis:7-alpine por valkey/valkey:8-alpine.
- **Beneficio:** licencia BSD-3 garantizada; performance similar o mejor.
- **Esfuerzo:** 1 hora (verificar compatibilidad con LiteLLM cache).
- **Riesgo:** bajo — drop-in replacement.

Next.js 15

- **Cambio:** `npm install next@15`, refactor de Route Handlers (async params), `next.config.ts` opcional.
- **Beneficio:** React 19, mejoras de cache, futuro asegurado.
- **Esfuerzo:** 3 días (refactor + testing).
- **Riesgo:** medio — breaking changes documentados pero numerosos.

17.2. Medio plazo (Q1-Q2 2027)

bge-m3 embedding

- **Cambio:** `BRAIN_EMBED_MODEL=BAAI/bge-m3` cuando fastembed lo soporte.
- **Beneficio:** +3-5 % MTEB multilingüe, soporte sparse + dense nativo.
- **Esfuerzo:** 4 días (re-indexación completa de Brain vault).
- **Riesgo:** bajo — dimensión sigue siendo 1024.

OpenTelemetry + Tempo

- **Cambio:** instrumentar el agente Python con OTel SDK; añadir Tempo como tracing backend; Grafana datasource para Tempo.
- **Beneficio:** traces distribuidos para queries lentas; correlación métricas-logs-traces.
- **Esfuerzo:** 5 días (instrumentación + dashboards).
- **Riesgo:** medio — overhead de tracing si no se samplea.

17.3. Largo plazo (H2 2027)

Multimodal input/output

- **Cambio:** pipeline de imagen (vision) en agente; soporte para Gemini 2.5 Vision como modelo default cuando hay imagen.
- **Beneficio:** usuarios pueden enviar fotos al agente para diagnóstico, contexto, ID.
- **Esfuerzo:** 2-3 semanas (UI + backend + storage).

WebRTC para voz

- **Cambio:** migrar voice channel de WebSocket a WebRTC con AEC / NS / VAD del browser.
- **Beneficio:** latencia E2E <200 ms, eco / ruido cancelados nativamente.
- **Esfuerzo:** 3-4 semanas (servidor WebRTC + cliente refactor).

Memoria episódica

- **Cambio:** añadir capa episódica sobre el grafo Kuzu (Letta-style hierarchical memory).
- **Beneficio:** agente responde "qué hablamos de X hace 2 semanas" con precisión.
- **Esfuerzo:** 4-6 semanas (diseño + implementación + evaluación).

Local LLM con GPU

- **Cambio:** montar GPU dedicada (RunPod / Hetzner GPU), correr vLLM con Llama 3.3 70B + LiteLLM como proxy.
- **Beneficio:** privacidad total + costo marginal \$0 (solo costo GPU mensual).
- **Esfuerzo:** 2-3 semanas (infra + monitoring).

18. Conclusiones

18.1. Veredicto general

Hermes Stack está **sólidamente alineado con el estado del arte** en 2026. Las decisiones tecnológicas reflejan un equilibrio entre:

- **Privacidad** (Whisper local, Brain self-hosted, sin SaaS observability).
- **Costo** (un único VPS, sin Kubernetes, sin DBs gestionadas).
- **Modularidad** (LiteLLM, MCP, Docker Compose — todas sustituibles).
- **Performance** (SLO <500 ms cumplido, p50 = 483 ms).

18.2. Las 4 decisiones que distinguen a Hermes positivamente

1. **LiteLLM como gateway LLM:** mejor que cualquier alternativa SaaS para self-hosting; el ecosistema lo confirma como SOTA.
2. **MCP / FastMCP desde el primer día:** apuesta temprana por el protocolo que se está convirtiendo en estándar.
3. **Brain con LanceDB + Kuzu + FastEmbed:** stack de memoria embedded sofisticado, sin servicios externos.
4. **Caddy + Docker Compose:** infraestructura minimalista que escala vertical hasta el límite del VPS.

18.3. Las 3 áreas con gap respecto al SOTA

1. **Observabilidad incompleta:** falta logs centralizados (Loki) y traces (Tempo + OTel) — esto importa cuando el sistema crezca.
2. **STT con latencia mejorable:** migrar a faster-whisper es low-hanging fruit (140 ms ganados).
3. **Privacidad de TTS:** ElevenLabs es el TTS premium pero los textos pasan por su API; necesario fallback Coqui para queries sensibles.

18.4. Las 2 apuestas a largo plazo

1. **Memoria episódica:** salir de RAG semántico puro y entrar en memoria con conciencia temporal. Diferenciará al sistema vs cualquier RAG genérico en el mercado.
2. **Multimodal con local LLMs:** cuando Llama 3.3 70B + GPU local sean estables, el costo marginal por query baja a \$0 y la privacidad sube a 100 %.

18.5. Mensaje de cierre

□ El Hermes Stack no es una colección de mejores prácticas en el vacío

Es una arquitectura cohesiva, diseñada deliberadamente para un caso de uso muy específico: **asistente IA personal autohospedado con voz**, controlado al 100 % por el propietario, sostenible con un único VPS por menos de \$50/mes. Esta restricción es la que justifica casi todas las decisiones, incluyendo las que se alejan del mainstream (Python manual vs LangChain, Compose vs K8s, Kuzu embedded vs Neo4j). Para ese caso de uso, el stack es competitivo con cualquier solución comercial — y ofrece algo que ninguna comercial puede: control y soberanía técnica completa.

19. Bibliografía y fuentes

19.1. Documentación oficial

Frameworks y librerías

- LangChain — python.langchain.com/docs
- Pydantic-AI — ai.pydantic.dev
- AutoGen (Microsoft) — microsoft.github.io/autogen
- CrewAI — docs.crewai.com
- DSPy (Stanford) — dspy-docs.vercel.app
- LiteLLM — docs.litellm.ai
- FastMCP — gofastmcp.com

Modelos y APIs

- Whisper paper — Radford et al., 2022 — arxiv.org/abs/2212.04356
- faster-whisper — github.com/SYSTRAN/faster-whisper
- ElevenLabs API docs — elevenlabs.io/docs
- OpenAI TTS — platform.openai.com/docs/guides/text-to-speech
- Coqui XTTS v2 — github.com/coqui-ai/TTS
- Kokoro TTS — huggingface.co/hexgrad/Kokoro-82M
- intfloat/multilingual-e5-large — huggingface.co/intfloat/multilingual-e5-large
- BAAI/bge-m3 — huggingface.co/BAAI/bge-m3
- Nomic Embed — nomic.ai/blog/posts/nomic-embed-text-v1.5

Bases de datos

- LanceDB docs — lancedb.github.io/lancedb
- Qdrant docs — qdrant.tech/documentation
- Chroma docs — docs.trychroma.com
- Weaviate docs — weaviate.io/developers/weaviate
- Kuzu docs — docs.kuzudb.com
- Neo4j docs — neo4j.com/docs
- pgvector — github.com/pgvector/pgvector

Infraestructura

- Caddy docs — caddyserver.com/docs
- Nginx docs — nginx.org/en/docs
- Traefik docs — doc.traefik.io
- Docker Compose — docs.docker.com/compose

- Kubernetes — kubernetes.io/docs
- Podman / Quadlet — docs.podman.io
- Prometheus — prometheus.io/docs
- Grafana — grafana.com/docs
- OpenTelemetry — opentelemetry.io/docs

19.2. Benchmarks y evaluaciones

- MTEB (Massive Text Embedding Benchmark) leaderboard — huggingface.co/spaces/mteb/leaderboard
- LMSys Chatbot Arena (LLM evaluation) — lmarena.ai
- Open LLM Leaderboard (Hugging Face) — huggingface.co/spaces/open-llm-leaderboard
- Artificial Analysis (LLM benchmarks) — artificialanalysis.ai
- Vector DB Bench (Zilliz) — github.com/zilliztech/VectorDBBench

19.3. Papers académicos relevantes

- **Whisper**: “Robust Speech Recognition via Large-Scale Weak Supervision” — Radford et al., OpenAI, 2022.
- **E5 embeddings**: “Text Embeddings by Weakly-Supervised Contrastive Pre-training” — Wang et al., Microsoft, 2022.
- **BGE M3**: “M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity” — Chen et al., BAAI, 2024.
- **RRF**: “Reciprocal Rank Fusion outperforms Condorcet and Individual Rank Learning Methods” — Cormack et al., 2009.
- **HNSW**: “Efficient and robust approximate nearest neighbor search using HNSW” — Malkov & Yashunin, 2018.
- **MCP**: “Model Context Protocol Specification” — Anthropic, 2024.
- **vLLM**: “Efficient Memory Management for Large Language Model Serving with PagedAttention” — Kwon et al., 2023.

19.4. Blogs y posts de referencia

- Simon Willison’s blog — simonwillison.net (análisis continuo de LLMs y agentes)
- Latent Space podcast — latent.space
- Phillip Carter’s blog — phillipcarter.dev
- Anthropic Engineering Blog — www.anthropic.com/engineering

19.5. Documentación interna del Hermes Stack

- `ficha_tecnica_hermes.pdf` — documento compañero (inventario técnico exhaustivo).
- `Hermes_Stack_Blueprint.pdf` — arquitectura v2.0 (52 pp).
- `Roadmap_Tecnico_Definitivo_v2.pdf` — roadmap completo.
- `Hermes_AutoMejora_2026-05-25.pdf` — informe de mejoras aplicadas.
- `informe-practicas-ia.pdf` — análisis crítico de patrones operativos.

20. Apéndice — benchmarks de referencia

20.1. Latencia E2E del pipeline de voz (50 muestras)

Etapa	p50 (ms)	p95 (ms)	p99 (ms)
Captura + Caddy + Next.js	27	45	78
Whisper STT (base)	180	245	310
LiteLLM cache lookup	5	12	28
Gemini Flash (cache miss)	280	420	580
Brain memory store (async)	0	0	0
ElevenLabs Flash v2.5 first chunk	75	105	145
Total E2E	483	712	1030

20.2. Distribución de cache hits LiteLLM (últimas 24 h)

Modelo	Requests	Cache hits
gemini-flash	920	312 (33.9 %)
gpt-4o-mini	215	68 (31.6 %)
llama-3.1-70b	87	29 (33.3 %)
claude-sonnet-4.6	18	2 (11.1 %)
gpt-4o	7	1 (14.3 %)
Total	1247	412 (33.0 %)

20.3. Consumo de recursos por contenedor (estable, 24 h)

Contenedor	CPU (avg %)	RAM (MB)	Net (MB/h)
hermes-agent	1.2	280	12
litellm-router	0.8	220	38
hermes-website	0.5	180	8
hermes-workspace	0.3	140	4
whisper-stt	2.5 (spikes)	850	6
redis-cache	0.2	35	18
brain	0.6	380	4
brain-worker	0.4	320	2
prometheus	0.4	180	28
grafana	0.2	110	5
cadvisor	1.1	95	16
node-exporter	0.1	25	12
9router	0.2	65	2
filebrowser	0.1	20	1
autoheal	0.1	15	1
Total	8.7 %	2.9 GB	157 MB/h

20.4. Costo operacional (estimación mensual)

Concepto	Costo USD	Notas
VPS Hostinger 4 vCPU / 8 GB	18	contrato anual
Dominio el80.space	1	anualizado
OpenRouter (LLMs)	13	1.2M tok in + 320k out por día
ElevenLabs (TTS)	5	300 conversaciones/mes
Hostinger DNS API	0	incluido
Tailscale (free tier)	0	hasta 100 devices
Total mensual	37 USD	

20.5. Disponibilidad histórica

Mes	Uptime	Incidentes
Enero 2026	99.92 %	1 (LiteLLM cuota)
Febrero 2026	99.78 %	2 (TLS renewal, Caddy reload)
Marzo 2026	99.97 %	1 (Brain Phase 5 deploy)
Abril 2026	99.99 %	0
Mayo 2026 (parcial)	100 %	0
Promedio	99.93 %	4 total

Fin del documento

Comparativa Hermes Stack vs Estado del Arte 2026
Compilado el 28 de mayo de 2026 · XeLaTeX · DejaVu Fonts
Para el inventario técnico detallado: ver
/root/docs/ficha_tecnica_hermes.pdf